

“මෙය අපගේ ER diagram එක. මේ diagram එකේ system එකේ data flow එක සහ table relationships පෙන්වනවා. Users table එක role-based access වලට use වෙනවා. Customers, Products, Product Variants සහ Orders tables sales process එකට directly connect වෙනවා. Order එකක් create වුණාම ඒකට Order Items attach වෙනවා, සහ Product Variant එකක් හරහා stock and price details manage කරනවා. මේ database design එක නිසා sales, inventory, quotation, delivery, notifications සහ reports වගේ modules එක centralized database එකකින් handle කරන්න පුළුවන්.”

“මේක අපේ system එකේ high-level architecture එක. Users browser හෝ mobile device එකේ React frontend එක access කරනවා. Frontend එක Axios API calls හරහා Node.js Express backend එකට communicate කරනවා. Backend එක business logic, validation, authentication, order processing සහ report generation handle කරනවා. Database side එක Sequelize ORM හරහා PostgreSQL tables manage කරනවා. ඒකට අමතරව Cloudinary image upload, Email OTP, WhatsApp notification, QR code, PDF generation වගේ services integrate කරලා system එක practical business environment එකකට suitable කරලා තියෙනවා.”

“මම project එකේ version control system එක විදිහට අපි GitHub use කලා. ඒකෙන් team members ලාගේ code changes track කරන්න, commits history බලන්න, bugs fix කරන්න සහ module-wise development manage කරන්න ලේසි වුණා. Development process එකේදී එක් එක් member තමන්ට assign වුණ tasks implement කරලා GitHub repo එකට push කලා. ඒ නිසා project එකේ progress එක transparent විදිහට track කරන්න පුළුවන් වුණා.”

“මේ slide එකේ අපේ personal GitHub contribution charts පෙන්වනවා. GitHub commits මගින් project එකේ individual progress සහ active development timeline එක track කරන්න පුළුවන් වුණා. මගේ contribution එක mainly user management, product management, order management, quotation, reports, logistics, authentication සහ offline order sync වගේ core system features වලට සම්බන්ධයි.”

“මේ slide එකේ project progress එක summarize කරනවා. අපි task-based development approach එකක් follow කලා. Tasks assign කරලා, in-progress සහ completed status වලින් track කලා. මගේ 143 tasks completed තියලා පෙන්වනවා. Pie chart එකේ team contribution distribution එක පෙන්වනවා. මේකෙන් project එක planned way එකකට progress වුණා තියලා demonstrate කරන්න පුළුවන්.”

“මම project එකේ documentation part එකක් අපි maintain කලා. SRS document එකේ functional සහ non-functional requirements define කලා. Database documentation වල ER diagram, schema සහ data dictionary include කලා. UI/UX design part එක Figma wireframes සහ prototypes වලින් support කලා. User guide එකේ role-based usage explain කලා. GitHub repository සහ ClickUp board එක project management documentation එකක් විදිහට use කලා.”

“Project management system එක විදිහට අපි ClickUp use කලා. ඒකෙන් tasks assign කරන්න, sprint-wise progress track කරන්න, in-progress සහ completed tasks බලන්න පුළුවන් වුණා. Team members ලාට තමන්ගේ tasks clearly identify කරන්න පුළුවන් වුණා. ඒ නිසා project timeline, workload සහ progress එක better manage කරන්න ClickUp ගොඩක් useful වුණා.”

## AxiosInstance usage

මමේ Axios configuration file එක use කරන්නේ frontend එකේ backend API වලට request යවන එක centralize කරලා manage කරන්න. මමේකෙන් API base URL එක set කරනවා, login වුණ user ගේ access token එක every request එකට automatically add කරනවා, token expire වුණොත් refresh token එක use කරලා new access token එකක් ගන්නවා, එහමෙ refresh කරන්න බැරි නම් userට logout කරනවා. ඒ නිසා API calls secure, reusable, and easy to manage වෙනවා.

## About Dexie

මමේ Dexie code එක use කරන්නේ application එකට offline support ලබාදීමයි. Dexie library එක භාවිතා කරලා browser එකේ IndexedDB database එකක් create කරනවා. ඒ database එකේ pending orders, customers, සහ products locally save කරගන්න පුළුවන්. Internet connection නැති වලොවක user කරන orders pendingOrders table එකේ store කරලා, connection ආවම backend එකට sync කරන්න මමේක උපකාරී වෙනවා.

## AuthContext

මමේ code එකේ purpose එක නමයි React app එකේ login/logout system එක manage කිරීම. It stores the logged-in user and access token, checks token expiry, refreshes the token automatically before it expires, and clears the session when logout or token failure happens.

## Why use migrations

“Migrations use කළේ database changes track කරන්න, team membersලා අතර same schema එක maintain කරන්න, deployment වලදී database structure එක automatically update කරන්න, සහ අවශ්‍ය නම් rollback කරන්න.”

## 1. භාවිතා කර ඇති පිරිමාන Libraries

frontend/package.json ගොනුවට අනුව, ඔබ භාවිතා කර ඇති පිරිමාන libraries සහ ඒවායේ කාර්යභාරය:

- **Core Framework:**
  - react, react-dom: පරිශීලක අනුප්‍රමුඛය (UI) ගොඩනැගීම සඳහා වන මූලික library එක.
- **Routing:**
  - react-router-dom: යඥුම තුළ විවිධ පිටු අතර සැරිසැරීම (client-side routing) සඳහා.
- **API Communication:**
  - axios: Backend API එක සමඟ ආරක්ෂිතව සහ පහසුවෙන් සන්නිවේදනය කිරීමට.
- **Styling & UI:**
  - tailwindcss: Utility-first CSS framework එකක්. වේගවත් සහ නිවැරදි UI නිර්මාණයට.
  - lucide-react: පැහැදිලි සහ සැහැල්ලු icon set එකක්.
  - framer-motion: UI elements සඳහා සුමට animations සහ transitions එක් කිරීමට.
- **Notifications & Alerts:**
  - react-hot-toast: කුඩා, නාවකාලික දැනුම්දීම (notifications) පෙන්වීමට.
  - sweetalert2: පරිශීලකයාගේ නහවුරු කිරීම අවශ්‍ය වන ලස්සන, interactive pop-up alerts සඳහා.
- **Offline Functionality:**
  - dexie: IndexedDB (browser එකේ ඇති database එකක්) සමඟ ඉතා පහසුවෙන් වැඩ කිරීමට. Offline order sync කිරීමේදී පිරිමාන කාර්යයක් ඉටු කරයි.

- **Document Generation:**
  - jspdf, html2canvas-pro, react-to-print: Reports, PDF ලෙස download කිරීමට සහ print කිරීමට.
- **QR Code Functionality:**
  - qrcode, react-qrcode: QR code generate කිරීමට සහ පෙන්වීමට.

## 2. භාවිතා කර ඇති තාක්ෂණික ක්රමවේද සහ Patterns

- **Component-Based Architecture:** React හි මූලික සිද්ධාන්තය වන, නැවත නැවත භාවිතා කළ හැකි components (උදා: ReportTable, ProductCard) වලට UI එක බඳවා වෙන් කර ඇත.
- **State Management (React Context API):**
  - AuthContext.jsx: User authentication සහ session එක කළමනාකරණය සඳහා Context එකක් නිර්මාණය කර ඇත. User ගේ තොරතුරු, token, login/logout functions මගින් අඩංගු වේ.
  - NotificationContext.jsx: යද්දම පුරා notifications කළමනාකරණය කිරීමට.
- **Proactive Token Refresh (ඉතා වැදගත් ආරක්ෂක ක්රමවේදයක්):**
  - AuthContext.jsx හි, access token එක expire වීමට පෙර (BUFFER\_TIME එකක් තබාගෙන), refreshToken එක භාවිතා කර ස්වයංකීර්ෂව අලුත් access token එකක් ලබාගැනීමේ ක්රියාවලියක් (refreshAccessTokenFn) නිර්මාණය කර ඇත.
  - setInterval එකක් මගින් මෙම ක්රියාවලිය නිරන්තරයෙන් පරීක්ෂා කිරීම නිසා, user හට session expired වි මෙමේ අපහසුතාවය මගහැරවේ. මෙය ඉතා දියුණු සහ ආරක්ෂිත ක්රමවේදයකි.
- **Offline-First Strategy (Dexie.js සමඟ):**
  - OfflineSyncManager.jsx මගින්, internet සම්බන්ධතාවය නොමැති විටදී පවා orders place කිරීමට හැකියාව ලබා දී ඇත.
  - මෙම orders, Dexie.js හරහා browser එකේ IndexedDB හි pendingOrders ලෙස ගබඩා කර, නැවත internet සම්බන්ධ වූ විට (navigator.onLine) ස්වයංකීර්ෂව backend එකට sync කරයි.
  - Products සහ Customers ලාගමේ දත්ත ද cache කරගැනීම නිසා offline search වැනි දේ කළ හැකි වේ.
- **Client-Side File Generation:**
  - **IIF Generation (Dashboard.jsx):** QuickBooks සඳහා .iif ගොනුවල බාහිර library එකක් නොමැතිව, JavaScript string manipulation මගින්ම නිර්මාණය කර ඇත. QB\_REF object එක මගින් ඔබගේ system එකේ දත්ත සහ QuickBooks හි දත්ත අතර mapping කිරීම (Adapter Pattern) ඉතා විශිෂ්ටයි.
  - **PDF Generation (Dashboard.jsx, LogisticsDashboard.jsx):** Mobile උපාංග සඳහා html2canvas-pro සහ jspdf භාවිතා කර, DOM එක clone කර, print-specific CSS යොදා, page-breaks පවා පාලනය කරමින් ඉතා උසස් තත්ත්වයේ PDF නිර්මාණය කිරීමේ ක්රියාවලියක් ඇත. මෙය සාමාන්ය react-to-print භාවිතයට වඩා බහෙවින් දියුණු ක්රමයකි.
- **QR Code Scanning & Generation:**
  - LogisticsDashboard.jsx හි, dispatch කිරීමේදී QR code scan කිරීමට html5-qrcode library එක භාවිතා කරයි.
  - Home.jsx හි, delivery confirmation සඳහා QR scan කර, backend එකෙන් OTP එකක් generate කරවා ගැනීමේ ක්රියාවලියක් ඇත.
  - LogisticsDashboard.jsx හි shipping label එක සඳහා QR code එක qrcode library එක මගින් canvas එකකට render කර, එය data URL එකක් ලෙස PDF එකට ඇතුළත් කරයි. මෙය SVG වලට වඩා html2canvas සමඟ නොදිව් ක්රියා කරයි.

Backend එක, ආරක්ෂාව, දත්තවල නිවැරදි බව (data integrity), සහ කාර්යක්ෂමතාවය ඉහළ මට්ටමක තබා ගනිමින් නිර්මාණය කර ඇත.

## 1. භාවිතා කර ඇති ප්රධාන Libraries

backend/package.json ගොනුවට අනුව, ඔබ භාවිතා කර ඇති ප්රධාන libraries:

- **Core Framework:**
  - express: Node.js සඳහා වන web application framework එක. API endpoints නිර්මාණය.
- **Database & ORM:**
  - sequelize: PostgreSQL database එක සමඟ සම්බන්ධ වීමට සහ queries ලිවීමට භාවිතා කරන ORM (Object-Relational Mapping) එක.
  - pg, pg-hstore: PostgreSQL සඳහා වන Node.js client එක.
- **Authentication & Security:**
  - jsonwebtoken: Access සහ Refresh token නිර්මාණයට සහ verify කිරීමට.
  - bcrypt: මුරපද (passwords) ආරක්ෂිතව hash කිරීමට.
  - cookie-parser: Browser cookies (විශේෂයෙන් HttpOnly refresh token එක) කියවීමට.
  - cors: Frontend එක සහ Backend එක අතර ආරක්ෂිත සන්නිවේදනයට.
- **File Uploads:**
  - multer, cloudinary, multer-storage-cloudinary: පින්තූර වැනි files, Cloudinary වැනි cloud storage සේවාවකට upload කිරීමට.
- **Emailing:**
  - nodemailer: Welcome emails, OTP, සහ invoices වැනි දේ email කිරීමට.
- **AI Integration:**
  - @google/generative-ai: Google Gemini API එක සමඟ සම්බන්ධ වීමට.
- **Development Tools:**
  - nodemon: Development කරන විට code එකේ වෙනසක් වූ සැනින් server එක restart කිරීමට.
  - dotenv: Environment variables කළමනාකරණයට.

## 2. භාවිතා කර ඇති තාක්ෂණික ක්රමවේද සහ Patterns

- **RESTful API Architecture:** සම්මත HTTP methods (GET, POST, PUT, DELETE) භාවිතා කරමින්, resource-based (users, products, orders) endpoints නිර්මාණය කර ඇත.
- **MVC (Model-View-Controller) වැනි ව්‍යුහයක්:**
  - **Models:** sequelize මගින් database tables සහ ඒවා අතර සම්බන්ධතා (associations) නිර්වචනය කර ඇත.
  - **Controllers:** එක් එක් endpoint එකට අදාළ business logic එක (උදා: userController, orderController) මගින් අඩංගු වේ.
  - **Routes:** API endpoint paths සහ ඒවාට අදාළ controller functions සම්බන්ධ කරයි.
- **Token-Based Authentication (JWT):**
  - authController.js හි, කෙටි කාලීන accessToken එකක් සහ දිගු කාලීන refreshToken එකක් භාවිතා කරන, ඉතා ආරක්ෂිත ක්රමවේදයක් ඇත.
  - refreshToken එක, JavaScript වලින් access කළ නොහැකි **HttpOnly cookie** එකක් ලෙස යැවීමෙන් Cross-Site Scripting (XSS) ප්රහාර වලින් ආරක්ෂාව වැඩි කර ඇත.
- **Middleware-based Authorization:**
  - authMiddleware.js හි verifyToken middleware එක මගින් protected routes ආරක්ෂා කරයි.
  - isAdmin, isAdminOrManager වැනි middleware මගින්, එක් එක් user role එකට අදාළව ක්රියාත්මක විය හැකි දේ (permissions) පාලනය කරයි (Role-Based Access Control - RBAC).
- **Advanced Database Operations (Sequelize):**
  - **Transactions:** orderController සහ userController වැනි තිරණාත්මක ස්ථානවල, sequelize.transaction භාවිතා කර ඇත. යම් ක්රියාවලියක කොටසක් fail වුවහොත්, සම්පූර්ණ operation එකම rollback කිරීම මගින් දත්තවල නිවැරදි බව (data integrity) සුරකිය.
  - **Soft Deletes:** Models වල paranoid: true යොදා තිබීම නිසා, records delete කරනවා වනුවට deleted\_at ලෙස සටහන් වේ. මෙය දත්ත නැවත ලබාගැනීමට සහ audit කිරීමට ඉතා වැදගත් වේ.

- **Eager Loading:** reportController.js හි මෙන්, include යොදාගෙන එකම query එකකින් අදාළ සියලුම tables (Order, Customer, User, Items) join කර දත්ත ලබාගැනීම, database performance වැඩි කරයි.
- **Data Encryption:**
  - cryptoUtils.js (inferred) හි ඇති encrypt/decrypt functions මගින්, phone numbers වැනි සංවේදී දත්ත database එකේ encrypt කර තැබීම, දත්ත ආරක්ෂාවට ඉතා වැදගත් පියවරකි.
- **Adapter Pattern:**
  - reportController.js හි getSalesReport function එකේදී, database එකේ ලැබෙන දත්ත (creator, OrderItems) ඒ ආකාරයෙන්ම frontend එකට නැවැත්වා, frontend එක බලාපොරොත්තු වන ආකාරයට (sales\_rep, items) structure එක වෙනස් කර යවයි. මෙය backend සහ frontend අතර ඇති coupling එක අඩු කරයි.

| Feature               | Used Library    | Purpose                                                                                                                                                                           |
|-----------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QR Scanner            | html5-qrcode    | Camera එකේ QR code scan කරන්න. මේක npm dependency එකක් විදහට නෙමෙයි, CDN script එකක් විදහට load කරනවා.                                                                            |
| Print                 | react-to-print  | React component එකක් print view එකට යවන්න. Shipping labels, reports, quotation print කරන්න use වෙනවා.                                                                             |
| PDF Generation        | jspdf           | PDF document create කරන්න.                                                                                                                                                        |
| HTML to Image/Canvas  | html2canvas-pro | HTML UI/component එක canvas image එකක් කරලා PDF එකට add කරන්න.                                                                                                                    |
| QR Code Generation    | qrcode          | Order ID එක QR code image/data URL එකක් කරලා generate කරන්න.                                                                                                                      |
| QR Code React Library | react-qr-code   | Package එක install කරලා තියනවා, QR generation related dependency එකක්. හැබැයි actual <b>LogisticsDashboard.jsx</b> එකේ current usage එකේ main use වන්නේ <b>qrcode</b> library එක. |
| Icons only            | lucide-react    | Scanner/print/PDF buttons වල icons display කරන්න.<br>Example: <b>ScanLine</b> , <b>QrCode</b> , <b>Printer</b> , <b>FileDown</b> .                                                |